

FRESHER ASSESSMENT PROJECT

Full-Stack Mini Web Application Challenge

Objective

Evaluate the candidate's ability to:

- Learn and apply new technologies quickly
- Build a simple but complete full-stack web application
- Package and run the app using Docker
- Explain how their code and components work internally

Task Overview

- Design & implement a simple web application with both frontend and backend.
- Containerize the full system using Docker.
- Explain the internal workflow and code logic (both architectural and runtime flow).

Suggested Problem Statements

You can pick any one of the following projects:

1. Mini Transaction Ledger

- Create accounts, record debit/credit entries, compute running balances.
- Simple REST backend (Spring Boot / .NET / Flask) + React/Angular frontend.
- Dockerized with Postgres/SQLite.

2. Employee Leave Tracker

- CRUD for employees, leaves, status (approved/pending).
- Add a REST auth layer (JWT).
- Frontend dashboards.

3. Mini Payment Router Simulator

- API to “quote” and “transfer” between dummy Digital Finance Service Providers.
- Basic request validation and logging to file.
- Docker multi-service setup.

Technical Requirements

- Frontend: Any modern framework.
- Backend: Any language or framework.
- Database: Optional — can use in-memory store, JSON file, SQLite, or any simple DB.
- Containerization:
 - The app must run inside Docker containers (frontend + backend).

- Use docker-compose (if multi-container setup).

Submission Requirements

1. Source Code Repository (GitHub/GitLab)

- Must include clear folder structure: /frontend, /backend, /docker-compose.yml
- Include README.md with:
 - Short project description
 - Tech stack used
 - Setup & run instructions
 - Explanation of architecture
 - Brief explanation of inner workings

2. Short Written Explanation (Required)

- A written document explaining:
 - App architecture (frontend-backend interaction)
 - Key code components and their purpose
 - How the API works internally
 - Docker setup explanation

Evaluation Criteria

Criteria	Weight	Description
Architecture Understanding	25%	Candidate can clearly explain code logic and data flow
Learning Ability	25%	How quickly and effectively new frameworks/tools are applied
Code Quality	20%	Readability, structure, and meaningful naming conventions
Functionality	15%	App fulfills basic requirements and runs successfully
Documentation & Presentation	10%	Clarity of README and explanation quality
Dockerization	5%	Proper use of Docker and/or docker-compose

Note:

- You can use AI, internet references or official documentation but must be able to explain all code and architecture decisions.

Deliverables Checklist

- ☐ Source code in GitHub with meaningful commits showing the development progress
- ☐ README with setup + architecture + explanation
- ☐ Docker file(s)
- ☐ docker-compose.yml (if applicable)